

TP1 – Operação Codebreaker: Documentação Técnica

1. Modelagem com Máquina de Estados Finitos (FSM)

Desde o início ficou claro que o fluxo do cliente e do servidor seguia uma sequência bem definida de passos — conectar, trocar mensagens, encerrar. Modelar isso como uma FSM pareceu natural: o loop principal de cada programa é um `while(1)` com um `switch(state)`, e cada `case` é um estado.

O servidor passa pelos seguintes estados:

```
START → SETTING_UP → WAIT_FOR_CONNECTION → WAIT_FOR_START_MESSAGE  
→ RECEIVED_START_MESSAGE → WAITING_FOR_MESSAGE → RECEIVED_GUESS  
→ SEND_FEEDBACK → [WAIT_FOR_EXIT | WAITING_FOR_MESSAGE] → EXIT
```

O cliente segue um fluxo parecido:

```
START → CONNECT_TO_SERVER → SEND_START_MESSAGE → SEND_GUESS  
→ WAIT_FOR_FEEDBACK → [IN_GAME | WIN | ERROR] → EXIT
```

Na prática, isso ajudou bastante na hora de depurar — quando algo dava errado, dava para saber exatamente em qual estado o programa estava e qual transição falhou. Também deixou o código mais legível, já que cada estado tem uma responsabilidade clara.

2. Bug no Preenchimento do Feedback

Esse foi o bug que mais custou tempo. O servidor calculava o feedback certo, mas o cliente recebia as dicas como `_ _ _ _ _` independentemente do palpite.

O problema: a `msg_to_send` era zerada com `memset` a cada rodada, mas eu esqueci de copiar o `guess` recebido para dentro dela antes de enviar. O cliente usava o `guess` da mensagem de resposta para montar a string de dicas — e como ele estava zerado, todos os dígitos apareciam como ausentes.

A correção foi chamar `fillFeedbackWithGuess` antes de calcular e enviar:

```
fillFeedbackWithGuess(msg_received.guess, &msg_to_send);  
calculateFeedback(msg_received.guess, code, &msg_to_send);
```

A lógica do cálculo em si estava correta desde o começo — o problema era só o preenchimento da struct.

3. Detecção de IPv4 ou IPv6 no Cliente

O servidor recebe o protocolo explicitamente via argumento (**v4** ou **v6**), mas o cliente recebe só o endereço IP. Para saber qual família usar, a solução foi tentar **inet_pton** para cada uma:

```
if (inet_pton(AF_INET, server_ip, &server_address_v4.sin_addr) == 1) {  
    // conecta via IPv4  
}  
if (inet_pton(AF_INET6, server_ip, &server_address_v6.sin6_addr) == 1) {  
    // conecta via IPv6  
}
```

É meio que um gambiarra, mas funciona bem: endereços IPv4 e IPv6 têm formatos incompatíveis, então **inet_pton** retorna 1 apenas para o formato correto. Não é necessário checar o protocolo explicitamente porque o próprio endereço já diz qual é.

4. Validação do Palpite: Duplicada de Propósito

A função **isValidGuess** existe em duas versões com assinaturas diferentes, e isso foi intencional.

- **Cliente** — **isValidGuess(const char *guess_string)**: valida a string crua lida do **stdin**, checando comprimento e se cada caractere é dígito com **isdigit**.
- **Servidor** — **isValidGuess(const int *guess)**: valida o array de inteiros já desserializado da mensagem, checando se cada valor está entre 0 e 9.

Cada uma vive numa fronteira diferente do sistema. O cliente valida antes de converter e enviar — é a barreira contra entrada ruim do usuário. O servidor valida depois de receber — é a barreira contra mensagens mal formadas vindas da rede, especialmente de clientes de outros alunos que podem não ter feito as mesmas verificações.

Unificar as duas não faria sentido porque operam em tipos diferentes e com objetivos diferentes. A duplicação aqui é consequência natural dessa separação de responsabilidades.

5. Funções de Leitura Separadas

readMessageFromClient no servidor e **readMessageFromServer** no cliente são estruturalmente iguais. Optei por mantê-las separadas porque cliente e servidor podem precisar de comportamentos diferentes no futuro — timeout, logging, tratamento de erros específico. Com funções separadas, qualquer mudança num lado não afeta o outro.

6. "Address already in use" e SO_REUSEADDR

Durante os testes, reiniciar o servidor logo depois de matar ele com Ctrl+C gerava um erro de **bind**: **Address already in use**. O kernel mantém o socket em **TIME_WAIT** por alguns segundos após o fechamento, bloqueando o reuso da porta.

A solução foi setar **SO_REUSEADDR** antes do **bind**:

```
int opt = 1;
setsockopt(server_socket, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

Isso diz pro kernel liberar a porta imediatamente, o que facilita bastante na hora de testar.

7. Endianness com htonl/ntohl

A `HackerMessage` é enviada como bytes crus pelo TCP. O problema é que arquiteturas diferentes armazenam inteiros em ordens de bytes distintas (*big endian* vs *little endian*), e conectar clientes e servidores de máquinas diferentes sem tratar isso pode gerar leituras completamente erradas.

Para resolver, as funções `messageHostToNetwork` e `messageNetworkToHost` convertem todos os campos `int` da struct antes de enviar e depois de receber:

```
msg->type      = htonl(msg->type);
msg->guess[i]   = htonl(msg->guess[i]);
msg->feedback[i] = htonl(msg->feedback[i]);
msg->attempts   = htonl(msg->attempts);
msg->win_status = htonl(msg->win_status);
```

O campo `char message[MSG_SIZE]` fica de fora porque bytes individuais não têm problema de endianness.

8. Leituras Parciais no TCP

TCP é um protocolo de stream — não tem garantia de que um `recv` retorna todos os bytes de uma vez. Com uma struct do tamanho de `HackerMessage`, isso é um problema real. A primeira versão usava um `recv` simples e funcionava nos testes locais, mas é frágil.

A solução foi um loop que acumula os bytes até completar o tamanho esperado:

```
while (total < sizeof(HackerMessage)) {
    int n = recv(socket, ((char*)msg) + total, sizeof(HackerMessage) -
total, 0);
    if (n <= 0) return ERROR;
    total += n;
}
```

O ponteiro avança pelo offset `total` a cada iteração, pedindo só os bytes que ainda faltam. Sem isso o programa poderia processar mensagens incompletas sem perceber.

9. Encerramento com MSG_EXIT

O encerramento segue um handshake simples de dois passos:

1. O **cliente**, ao receber `win_status == WIN`, exibe a mensagem de vitória, envia `MSG_EXIT` e fecha o socket.
2. O **servidor**, ao mandar o feedback de vitória, vai para `WAIT_FOR_EXIT_MESSAGE_STATE` e só fecha os sockets depois de receber o `MSG_EXIT`.

Isso evita que o servidor feche a conexão enquanto o cliente ainda está processando a mensagem final.

10. Servidor Iterativo

O servidor aceita uma conexão por vez, joga até o fim e volta a aguardar. Sem `fork`, `threads` ou `select`.

Para o escopo do TP isso é suficiente — o enunciado não pede múltiplos clientes simultâneos, e um servidor concorrente adicionaria complexidade desnecessária aqui. É provável que esse seja exatamente o ponto do TP2.

11. scanf vs fgets

A leitura do palpite começou com `scanf`, mas trocou para `fgets`. O `scanf` com `%s` para no primeiro espaço e deixa o `\n` no buffer, o que quebrava as leituras seguintes. O `fgets` lê a linha inteira e o `\n` fica dentro da string — o que é fácil de tratar com `strcspn` na hora de checar o comprimento.